GuardCode: Suppressing Undetected Frame Errors with a Soft-Syndrome Guard and a Learned Gate on CRC-Protected Links

Project Report
BCSE308L - Computer Networks
BCSE308P - Computer Networks Lab

Submitted by
S Naresh Nireenjan (24BCE2353)
Dharshan J (24BCE2335)
Sachin Saravanan S (24BCE2321)

Under the guidance of
Dr. Anusha N

School of Computer Science and Engineering
Vellore Institute of Technology, Vellore
October, 2026

Contents

Chapter	Covers
Abstract, 1. Introduction	Context and problem statement
2. Research Gap	Related work and the gaps this project fills
3. Objectives	Objectives, each mapped to a gap and to where it is achieved
4. Workflow and Methodology	Workflow diagram, system architecture, methods
5. Code	Software modules, key code listings, database design
6. Output Implementation	The running system (dashboard screenshots) and generated outputs
7. Comparison Tables	Receivers, channels, CRC-8 vs CRC-16, existing methods
8. Conclusion and Future Work, References, Appendix	Summary and how to run

Abstract

Every frame on a data link carries a cyclic redundancy check (CRC). The receiver decodes the frame, recomputes the CRC, and either delivers the frame or asks for a retransmission. The weak spot is the **undetected error**: a frame that is decoded wrongly but still satisfies the CRC, so corrupted data is delivered silently. This project builds **GuardCode**, a receiver that removes much of that gap without adding a bit to the frame and without changing the CRC.

Its core, the **Soft-Syndrome Guard (SSG)**, uses the decoder's soft output and the exact algebra of the standard CRC to compute, for every frame that passes the CRC, the probability that it is still wrong; risky frames are retransmitted. A gradient-boosted machine-learning gate (**GuardCode-ML**) refines that probability. With 3,00,000 simulated frames per operating point and the same retransmission budget for every receiver, SSG removed 48 % (AWGN) and 50 % (burst) of the undetected errors left by the classic confidence test, and GuardCode-ML removed a further 56 % and 24 %. On a channel built from 22 **real measured 802.15.4 error traces** (CRAWDAD niit/bit_errors), with nothing re-tuned, SSG still left 50 % fewer undetected errors than the classic test. Everything works with CRC-8 and CRC-16. A live web dashboard and a SQLite database that logs every frame show the system running.

Keywords: CRC, undetected error, ARQ, forward error correction, neural channel code, machine learning, Hamming code, burst errors, data link layer.

1. Introduction

1.1 Background

Module 3 of the course covers error detection and correction (parity, checksum, CRC, Hamming code) and flow and error control (Stop-and-Wait, Go-Back-N and Selective-Repeat ARQ). In a real link these work together: a forward error correction (FEC) code repairs most channel errors, the CRC decides whether the repaired frame can be trusted, and ARQ retransmits the frames the CRC rejects.

A CRC with r check bits misses about one in 2^r of the error patterns that reach it. For the short CRCs used in IoT, sensor and control traffic (8 to 16 bits) this is not negligible: one undetected error per few thousand frame errors can corrupt a sensor stream or a control command.

1.2 Problem statement

Can a receiver lower the undetected-error rate of a CRC-protected link, at a fixed retransmission budget, without extra redundancy and without changing the CRC or the frame format?

1.3 Mapping to the syllabus

Course topic	Where it appears in this project
Module 3 - CRC (polynomial division)	guardcode/crc.py: CRC-8 and CRC-16, generator and parity-check matrices
Module 3 - Hamming code	guardcode/hamming.py: Hamming(7,4), soft decoding, block interleaver
Module 3 - Burst errors, interleaving	Gilbert-Elliott burst channel; real 802.15.4 burst traces; interleaved profiles
Module 3 - ARQ (Selective Repeat)	Goodput model; NAK on CRC or guard rejection

Course topic	Where it appears in this project
Module 1 - Transmission impairments	AWGN, burst noise, Rayleigh fading and measured channels
Module 6 - QoS, reliability	Undetected error rate as a QoS metric; risk ledger
Lab - socket / web programming	Flask web server, Server-Sent Events, REST API, SQL explorer

Table 1 - Syllabus mapping

2. Research Gap

2.1 Related work

Work	Idea	Limitation
CRC polynomial design for a given inner code (Lou, Wesel et al., 2015)	Choose g(x) to minimise undetected errors for a convolutional code	Requires changing the standard polynomial
CRC-aided list decoding (polar CA-SCL; US 2023/0198672 A1)	CRC selects among decoder candidates	Decoder-specific; no estimate of the accepted frame's residual risk
Neural channel codes (Turbo Autoencoder; US 12,155,482 B2)	Codes learned end-to-end	Use parity or LLR checks only as runtime flags
Erasure / reliability tests (Forney 1968; ROVA 1998; polar UER thresholds, 2023)	Reject when decoder confidence is low	Confidence is blind to which errors the CRC would catch
Gallager's parity-check probability (LDPC, 1963)	Probability that a parity check is satisfied	Used inside iterative decoders, not as a CRC acceptance test
Wireless error measurements (CRAWDAD niit/bit_errors, 2008)	Real residual bit-error traces of 802.15.4 and 802.11	Characterise errors; not used to test acceptance rules

Table 2 - Related work

2.2 Gaps addressed by this project

Gap	Description	How this project addresses it
G1	Receivers treat the CRC as a pass/fail bit; no method gives the probability that a CRC-passing frame is wrong from the standard CRC alone	Soft-Syndrome Guard: exact CRC-conditioned posterior via the dual-code expansion
G2	Confidence thresholds waste the retransmission budget on errors the CRC would catch anyway	Gate only on the risk that the CRC cannot see, at an equal retransmission budget
G3	Lowering undetected errors usually needs more CRC bits or a non-standard polynomial	No extra bits; unmodified CRC-8 / CRC-16
G4	Neural-decoder probabilities are not perfectly calibrated, so an exact formula alone is not optimal	GuardCode-ML: a learned gate that uses the exact risk as one input
G5	Acceptance rules are evaluated on idealised simulated channels only	Validation on a channel fitted to 22 real measured 802.15.4 error traces
G6	Receivers cannot tell how many corrupted frames they delivered	Risk ledger: running estimate without ground truth

Table 3 - Research gaps

A search of Google Patents, Justia and arXiv found no document that computes the CRC-conditioned posterior probability of an accepted frame being wrong through the dual-code expansion of a standard CRC and uses it as an ARQ acceptance gate.

3. Objectives

#	Objective	Gap	Achieved in
O1	Build a link simulator with CRC-8 / CRC-16 framing, Hamming(7,4) and learned neural FEC, AWGN, burst, fading and measured channels, and ARQ accounting	-	Ch. 4, 5
O2	Design a receiver test that computes the exact residual risk of each CRC-passing frame (Soft-Syndrome Guard)	G1, G3	Sec. 4.4, 5.2
O3	Compare it fairly with plain CRC acceptance and the min-|LLR| test at equal retransmission cost	G2	Ch. 7
O4	Add a learned (ML) acceptance gate and test whether it improves on the analytic guard	G4	Sec. 4.6, 7.1
O5	Validate on real measured error traces and support both CRC-8 and CRC-16	G5	Sec. 4.9, 7.1, 7.4
O6	Add a risk ledger, channel-adaptive profiles, a live dashboard and a frame database	G6	Sec. 4.7-4.8, Ch. 6

Table 4 - Objectives

4. Workflow and Methodology

4.1 Workflow

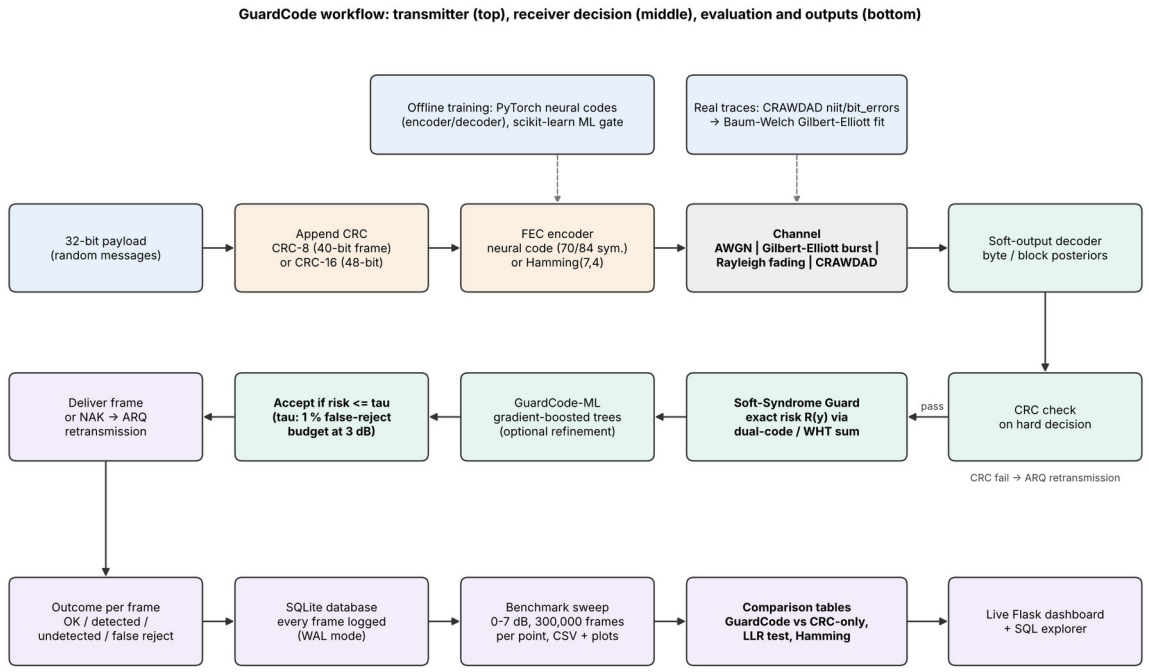


Figure 1 - End-to-end workflow

1. Generate a random 32-bit payload and append the CRC (CRC-8: 40-bit frame; CRC-16: 48-bit frame).
2. Encode with the FEC code: the learned neural code (70 or 84 symbols) or Hamming(7,4).
3. Pass through the channel: AWGN, Gilbert-Elliott burst, Rayleigh fading, or the channel fitted to real CRAWDAD traces.

4. Decode with a soft-output decoder, which gives a probability distribution for every byte or block.
5. Check the CRC on the hard decision. If it fails, request a retransmission.
6. If it passes, compute the Soft-Syndrome Guard risk R(y) and, optionally, the GuardCode-ML score.
7. Deliver the frame only if the score is below a threshold set for a 1 % false-reject budget; otherwise send a NAK.
8. Record the outcome (OK, detected, undetected, false reject) in the database, aggregate benchmark results, and build the comparison tables and the live dashboard.

4.2 System architecture

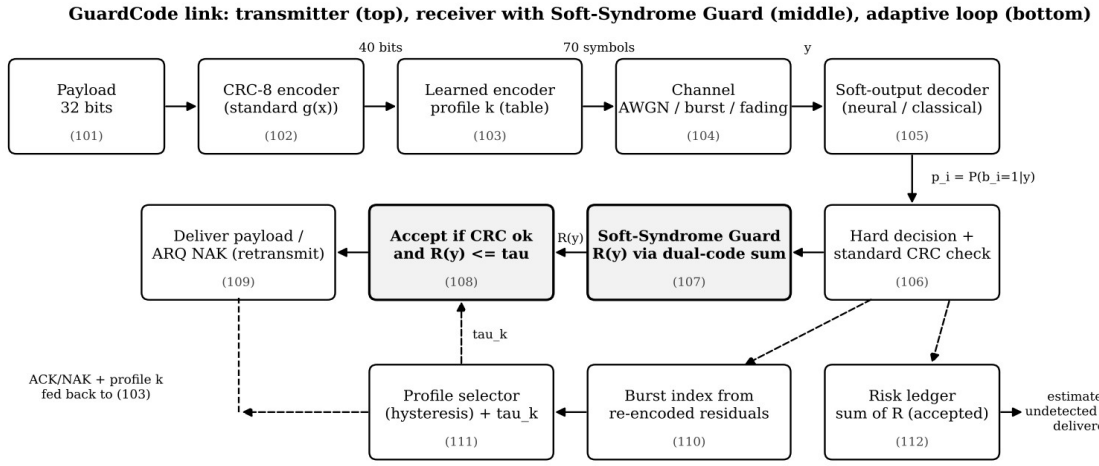


Figure 2 - System architecture

4.3 Frame format and CRC as a linear code

A frame carries 32 payload bits and an r-bit CRC: CRC-8 with g(x) = x^8 + x^2 + x + 1, or CRC-16 with g(x) = x^16 + x^12 + x^5 + 1 (CRC-16/CCITT, as in HDLC, X.25 and Bluetooth). The CRC is linear, so crc(m) = m G (mod 2), where row i of G is the CRC of the i-th unit vector, which is exactly the long division done in the lab. Every valid frame b satisfies H b = 0 with H = [G^T | I_r]. Exhaustive search gives a minimum distance of 4 for the 40-bit CRC-8 frame and 4 for the 48-bit CRC-16 frame.

4.4 Soft-Syndrome Guard

The decoder outputs a posterior over each **group** of frame bits: 256 values per byte for the neural decoder, 16 messages per 4-bit Hamming block, or one bit for uncoded BPSK. The hard decision x takes the most likely value of every group. Since the transmitted frame must be a CRC codeword, Bayes' rule gives:

$$P(b = x \mid y, Hb = 0) = \prod_c P_c(x_c) \,/\, P(Hb = 0 \mid y)$$

The denominator is computed exactly by summing over the 2^r words u of the dual code, where each group contributes one Walsh-Hadamard transform (WHT) coefficient of its posterior:

$$P(Hb = 0 \mid y) = 2^{-r} \cdot \sum_u \prod_c F_c(s_c(u)), \quad F_c(s) = \sum_v P_c(v)\,(-1)^{\mathrm{popcount}(s \,\mathrm{AND}\, v)}$$

The residual risk is R(y) = 1 − P(b = x | y, Hb = 0). A frame is delivered only if the CRC passes and R(y) ≤ τ. CRC-8 needs 256 terms per frame; CRC-16 needs 65,536, computed by a compiled

multi-core kernel at about 7,000 frames per second. The formula was checked against Monte-Carlo estimates: 0.003671 analytical vs 0.003663 simulated over 400,000 trials.

Worked example. Three bits with an even-parity rule; the decoder says each bit is 1 with probability 0.9, 0.8 and 0.3. The decision 110 passes and has probability 0.504. All parity-valid words together have probability 0.596, so R = 1 − 0.504/0.596 = 15.4 % and the frame is retransmitted. With 0.99, 0.98 and 0.01 the risk is 0.05 % and the frame is accepted.

4.5 Learned neural channel code

- **Encoder:** each byte (256 values) is mapped by a trainable table to 14 real symbols: 70 symbols per CRC-8 frame, 84 per CRC-16 frame. The burst profile adds a fixed interleaver.
- **Decoder:** a 3-layer MLP (512 ELU units per layer) sees all received symbols and outputs a 256-way posterior for every byte.
- **Training:** 5,000 Adam steps, batch 1,024, Eb/N0 uniform in 1-6 dB, through a differentiable channel in PyTorch; weights are exported so inference runs on numpy.

4.6 Learned risk gate (GuardCode-ML)

SSG is an exact posterior only if the decoder's probabilities are calibrated, and a neural decoder is only approximately calibrated. GuardCode-ML is a gradient-boosted decision-tree classifier (scikit-learn HistGradientBoostingClassifier) trained on 6 million simulated frames per channel profile to predict whether a CRC-passing frame is wrong. Its inputs are the SSG risk, the minimum |LLR|, sorted per-byte confidences and runner-up gaps, the residual noise energy after re-encoding the decision, and a burst index. Its threshold uses the same 1 % budget; training and test seeds are disjoint. No ML gate is trained for CRC-16: only about 2 in 600,000 CRC-16 frames are wrong but pass the CRC, too few examples to learn from.

4.7 Channel-adaptive profile controller

Two profiles are trained (AWGN and burst). The receiver re-encodes its decision, measures residual energy in windows of 7 symbols, and smooths log10(max/median) of those energies (EWMA 0.02). With hysteresis (switch up at 0.44, down at 0.40) it selects the profile and feeds the choice back as 1 bit on the ACK.

4.8 Risk ledger

Summing R(y) over delivered frames gives the receiver its own estimate of how many undetected errors it delivered, without knowing the transmitted data.

4.9 Real-world channel from measured traces (CRAWDAD)

The burst model was fitted to the public **CRAWDAD niit/bit_errors** dataset (IEEE DataPort, DOI 10.15783/C7S88K): residual bit-error traces of IEEE 802.15.4 MicaZ motes, 20-byte frames, 5-12 m from a base station in four indoor setups. The 22 traces hold 6,50,882 frames (10,41,41,120 payload bits). In a trace, a payload byte 0xAA means "correct" and an XOR with 0xAA gives the bits in error. A two-state Gilbert-Elliott hidden Markov model is fitted to every trace by the Baum-Welch algorithm (checked on synthetic data, where it recovers the planted parameters to within a few percent). Errors come in short bursts: the channel leaves the Bad state with probability about 0.24 per bit, so a burst lasts about 4 bits, and bits inside a burst are essentially random, matching one wrong 4-bit 802.15.4 symbol. Every simulated frame draws one measured trace's model. The

codes, thresholds and ML model are taken unchanged from the burst profile, so this is a genuine test of generalisation.

5. Code

5.1 Software modules

Component	File	Technology
CRC-8 / CRC-16, parity-check and dual-code matrices	guardcode/crc.py	Python, numpy
Channels (AWGN, Gilbert-Elliott, Rayleigh, CRAWDAD)	guardcode/channels.py	numpy; torch for training
Hamming(7,4) soft decoder, interleaver	guardcode/hamming.py	numpy
Neural code: training / inference	guardcode/model.py, train.py	PyTorch / numpy
Soft-Syndrome Guard	guardcode/guard.py	numpy; numba kernel for CRC-16
Receiver line-up and acceptance rules	guardcode/schemes.py	numpy
Learned risk gate (GuardCode-ML)	guardcode/mlguard.py, scripts/train_mlguard.py	scikit-learn
CRAWDAD loader, Baum-Welch fit	guardcode/crawdad.py, scripts/fit_crawdad.py	numpy, numba
Adaptive controller	guardcode/adaptive.py	numpy
Link simulator, benchmark, plots	guardcode/linksim.py, evaluate.py	numpy, matplotlib
Database layer	guardcode/db.py	SQLite (WAL mode)
Live dashboard and SQL explorer	dashboard/app.py, engine.py, templates	Flask, Server-Sent Events, Chart.js

Table 5 - Software modules

5.2 Key code

CRC encoding and checking (guardcode/crc.py)

```
    def encode(self, msgs):
        """msgs: (B, k) 0/1 -> frames (B, k+r) with CRC appended."""
        msgs = np.asarray(msgs, dtype=np.uint8)
        crc = (msgs.astype(np.int64) @ self.G) % 2
        return np.concatenate([msgs, crc.astype(np.uint8)], axis=1)

    def syndrome(self, frames):
        frames = np.asarray(frames, dtype=np.int64)
        return (frames @ self.H.T) % 2

    def check(self, frames):
        """True where the CRC passes."""
        return ~self.syndrome(frames).any(axis=1)
```

Soft-Syndrome Guard risk (guardcode/guard.py)

```
    def risk_groups(self, P, rows=None):
        """P: (B, G, 2^m) group posteriors. Returns (decision values (B,G), risk
(B,)).
        rows: optional boolean mask; the risk is computed only there (1.0 elsewhere).
The
        receiver needs it only for CRC-passing frames, which saves most of the CRC-16
cost."""
```

```
        P = np.asarray(P, dtype=np.float64)
        P = P / P.sum(axis=2, keepdims=True)
        dec = P.argmax(axis=2)
        risk = np.ones(P.shape[0])
        sel = np.arange(P.shape[0]) if rows is None else np.flatnonzero(rows)
        if sel.size:
            risk[sel] = self._risk(P[sel], dec[sel])
        return dec, risk

    def _risk(self, P, dec):
        B, G, M = P.shape
        m = int(np.log2(M))
        log_px = np.log(np.maximum(np.take_along_axis(P, dec[..., None], axis=2)[...,
0], 1e-300)).sum(axis=1)
        F = fwht(P)                                     # (B, G, M)
        idx = self._mask_index(m)                       # (U, G)
        U = idx.shape[0]
        if m == 1:
            # single-bit groups: prod_i t_i^{W_ui} with t = 1 - 2p, via log-magnitude
/ sign matmuls
            t = F[:, :, 1]
            Wf = idx.astype(np.float64).T                  # (G, U)
            mag = np.exp(np.log(np.maximum(np.abs(t), 1e-300)) @ Wf)
            sign = 1.0 - 2.0 * np.mod((t < 0).astype(np.float64) @ Wf, 2)
            pz = (sign * mag).mean(axis=1)
        elif _dual_sum is not None and U >= 4096:      # CRC-16; CRC-8 keeps the
numpy path
            pz = _dual_sum(np.ascontiguousarray(F), self._idx32(m))
        else:
            pz = np.empty(B)
            for a in range(0, B, _CHUNK):                # product over groups, one
gather each
                acc = np.ones((min(_CHUNK, B - a), U))
                for c in range(G):
                    acc *= F[a:a + _CHUNK, c, :][:, idx[:, c]]
                pz[a:a + _CHUNK] = acc.mean(axis=1)
        pz = np.maximum(pz, np.exp(log_px))            # numerical guard: P(code) >=
P(x)
        return np.clip(1.0 - np.exp(log_px) / pz, 0.0, 1.0)
```

Receiver decision: CRC, guard and gates (guardcode/schemes.py)

```
    def decide(self, y, h, ebno_db):
        """Returns bits, accept-mask (CRC + gate), crc_ok, score (risk or min|
LLR|)."""
        P = self.groups(y, h, ebno_db)
        B, G, M = P.shape
        m = int(np.log2(M))
        c = crc(self.crc_name)
        bits = _VALUE_BITS[m][P.argmax(axis=2)].reshape(B, G * m)
        crc_ok = c.check(bits)
        if c.r <= 8:
            _, risk = ssg(self.crc_name).risk_groups(P)
        elif self.gate in ("ssg", "ml"):
            # CRC-16: the 65,536-term risk is only needed where the CRC passes (1.0
elsewhere)
            _, risk = ssg(self.crc_name).risk_groups(P, rows=crc_ok)
        else:
            risk = np.full(B, np.nan)       # CRC-16 receivers without a guard: not
computed
        p1 = np.clip(P @ _VALUE_BITS[m].astype(np.float64), 1e-15, 1 - 1e-
15).reshape(B, G * m)
        llr = np.abs(np.log(p1) - np.log1p(-p1)).min(axis=1)
        accept = crc_ok.copy()
        extra = {}
        if self.gate == "llr":
            accept &= llr >= self.tau
        elif self.gate == "ssg":
```

```
            accept &= risk <= self.tau
        elif self.gate == "ml":
            extra["ml"] = ml_score = self.ml.score(mlguard.features(self, P, risk,
llr, y, h, bits))
            accept &= ml_score <= self.tau
        return bits, accept, crc_ok, dict(llr=llr, risk=risk, p1=p1, **extra)
```

GuardCode-ML features (guardcode/mlguard.py)

```
def features(scheme, P, risk, llr, y, h, bits):
    """P: group posteriors (B, G, 2^m); y, h: received symbols and channel gain;
bits: decisions."""
    B, G, M = P.shape
    Ps = np.sort(P, axis=2)
    pmax, p2 = Ps[:, :, -1], Ps[:, :, -2]
    conf = np.sort(np.log10(np.maximum(1.0 - pmax, 1e-12)), axis=1)[:, ::-1]      #
least confident first
    gap = np.sort(np.log(np.maximum(pmax, 1e-300)) - np.log(np.maximum(p2, 1e-300)),
axis=1)
    x_hat = scheme.encode(bits.astype(np.float64))
    r2 = (y * h - x_hat) ** 2
    n = r2.shape[1]
    w = r2[:, : (n // WINDOW) * WINDOW].reshape(B, -1, WINDOW).mean(axis=2)
    bidx = np.log10(w.max(axis=1) / (np.median(w, axis=1) + 1e-9) + 1e-9)
    return np.column_stack([np.log10(np.maximum(risk, 1e-12)), llr, conf, gap,
                            np.log10(r2.mean(axis=1) + 1e-9), bidx])
```

CRAWDAD trace loader (guardcode/crawdad.py)

```
def load_trace(path):
    """Returns the (frames, 160) 0/1 matrix of payload bit errors and the number of
skipped lines."""
    rows, skipped = [], 0
    with open(path) as f:
        for line in f:
            tok = line.split()
            if len(tok) != HEADER + PAYLOAD:
                skipped += bool(tok)
                continue
            rows.append([int(x, 16) for x in tok[HEADER:]])
    data = np.array(rows, dtype=np.uint8).reshape(-1, PAYLOAD)
    return np.unpackbits(data ^ 0xAA, axis=1), skipped
```

The complete source code is in the project repository; Appendix A lists the commands to run it.

5.3 Database design

Every transmitted frame, from bulk runs and from live operation, is stored as one row. WAL (write-ahead logging) mode lets the live simulator write over 10,000 rows per second while the dashboard reads. Setup populates 1,000,000 frames; a retention policy keeps the newest 10 million rows during live use.

Table	Key columns	Purpose
frames	run_id, ts, seq, scheme, channel, ebno_db, bit_errors, outcome, reliability, attempt, crc	One row per frame (outcome: 0 OK, 1 detected, 2 undetected, 3 false reject)
runs	id, started_at, mode, notes	Live sessions, bulk populations, benchmarks
benchmark	profile, crc, scheme, ebno_db, frames, fer, uer, goodput ...	Aggregated Monte-Carlo results
adaptive_log	ts, burst_index, chosen_profile, true_channel	Decisions of the adaptive controller
v_summary (view)	scheme, channel, ebno_db, counts	Per-scheme summary

Table 6 - Database schema (data/guardcode.db)

6. Output Implementation

6.1 Simulation setup

Parameter	Value
Payload / CRC / frame	32 bits / CRC-8 / 40 bits; or CRC-16 / 48 bits
Channel symbols per frame	CRC-8: 70 coded, 40 uncoded; CRC-16: 84 coded, 48 uncoded
Eb/N0 range	0 to 7 dB (energy per information bit)
Burst channel	Gilbert-Elliott: P(G->B) = 0.03, P(B->G) = 0.25, +8 dB noise in Bad state
Measured channel	Per-trace Gilbert-Elliott mixture fitted to 22 real 802.15.4 traces
Frames per operating point	3,00,000
Gate calibration	Every gate rejects 1 % of correctly decoded frames at 3 dB (equal ARQ cost)
ARQ model	Selective repeat; goodput = correctly delivered info bits per channel use

Table 7 - Simulation parameters

6.2 Live dashboard

The running system is a Flask web application. A background thread transmits frames continuously through the selected channel with all eight receivers side by side, writes every frame to the database and pushes a snapshot to the browser twice a second. The operator controls Eb/N0, the channel (AWGN, burst, fading, measured 802.15.4), the CRC (CRC-8 / CRC-16), the frame rate and a 20-second burst storm.

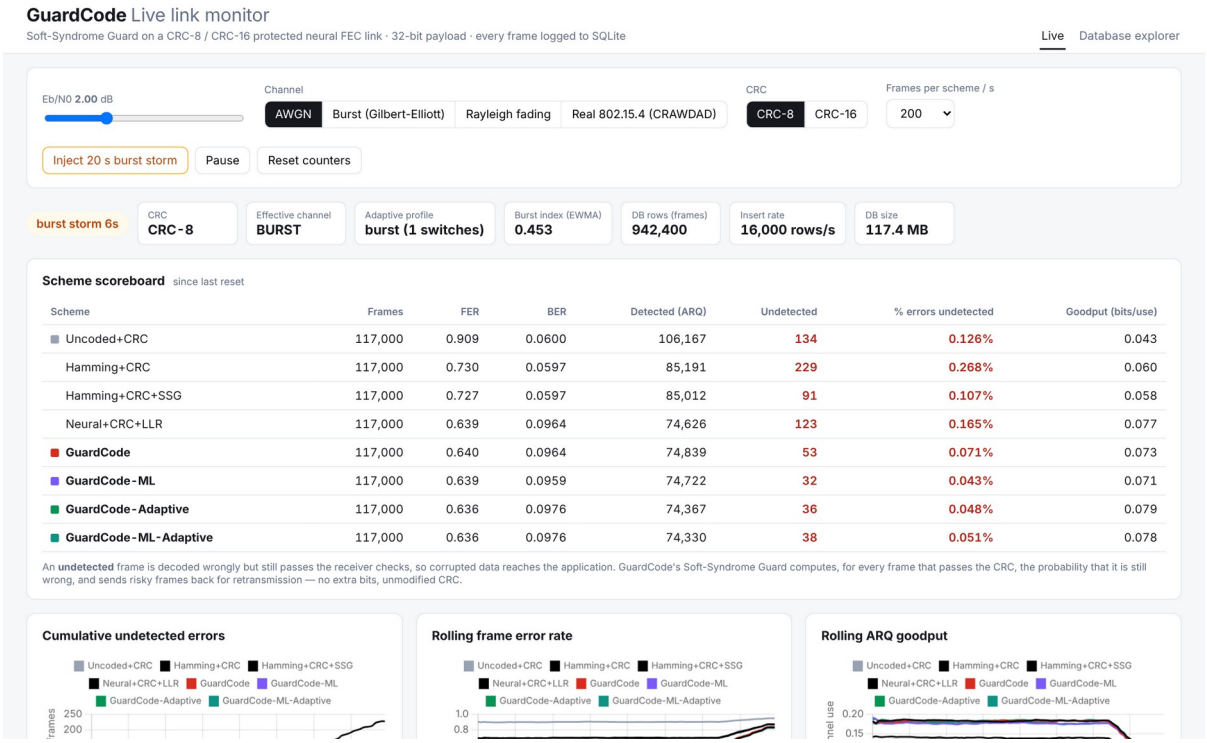
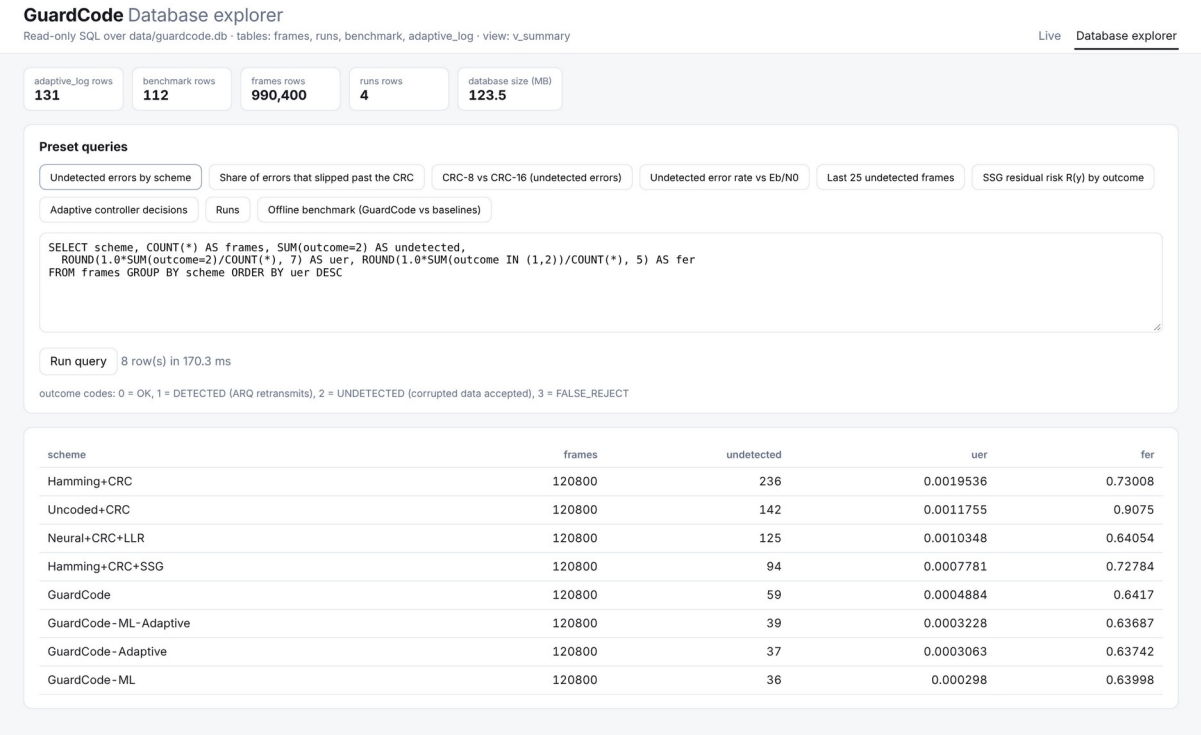


Figure 3 - Live dashboard during a burst storm at Eb/N0 = 2 dB (117,000 frames per receiver)

In the run shown in Figure 3, the classic confidence test (Neural+CRC+LLR) let 123 corrupted frames through, GuardCode 53 and GuardCode-ML 32, while all neural receivers had about the same frame error rate and goodput. The adaptive receivers switched to the burst profile during the storm (burst index 0.453 above the 0.44 threshold).



scheme	frames	undetected	uer	fer
Hamming+CRC	120800	236	0.0019536	0.73008
Uncoded+CRC	120800	142	0.0011755	0.9075
Neural+CRC+LLR	120800	125	0.0010348	0.64054
Hamming+CRC+SSG	120800	94	0.0007781	0.72784
GuardCode	120800	59	0.0004884	0.6417
GuardCode-ML-Adaptive	120800	39	0.0003228	0.63687
GuardCode-Adaptive	120800	37	0.0003063	0.63742
GuardCode-ML	120800	36	0.000298	0.63998

Figure 4 - SQL explorer: undetected errors per receiver, queried from about one million logged frames

6.3 Benchmark outputs

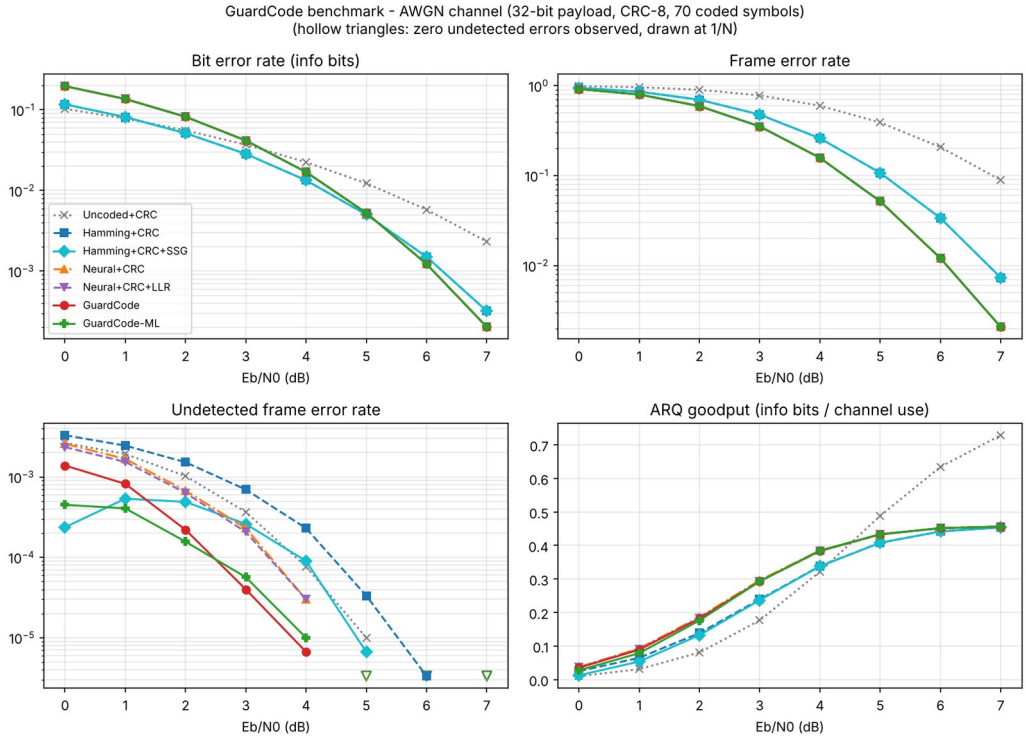


Figure 5 - AWGN channel: BER, FER, undetected FER and goodput (CRC-8)

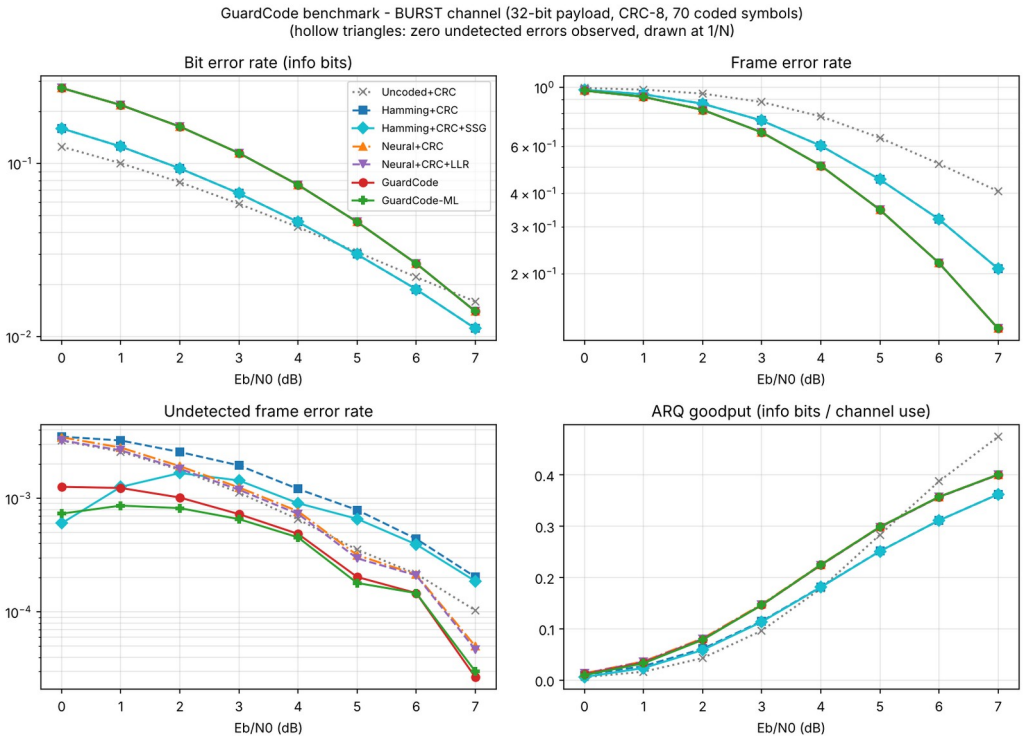


Figure 6 - Gilbert-Elliott burst channel (CRC-8)

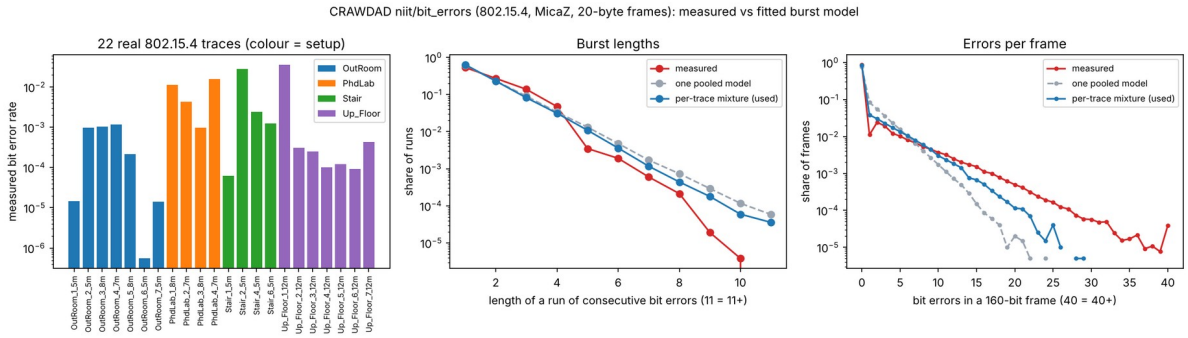


Figure 7 - The 22 measured CRAWDAD traces: error rate per trace, burst lengths and errors per frame, measured vs fitted model

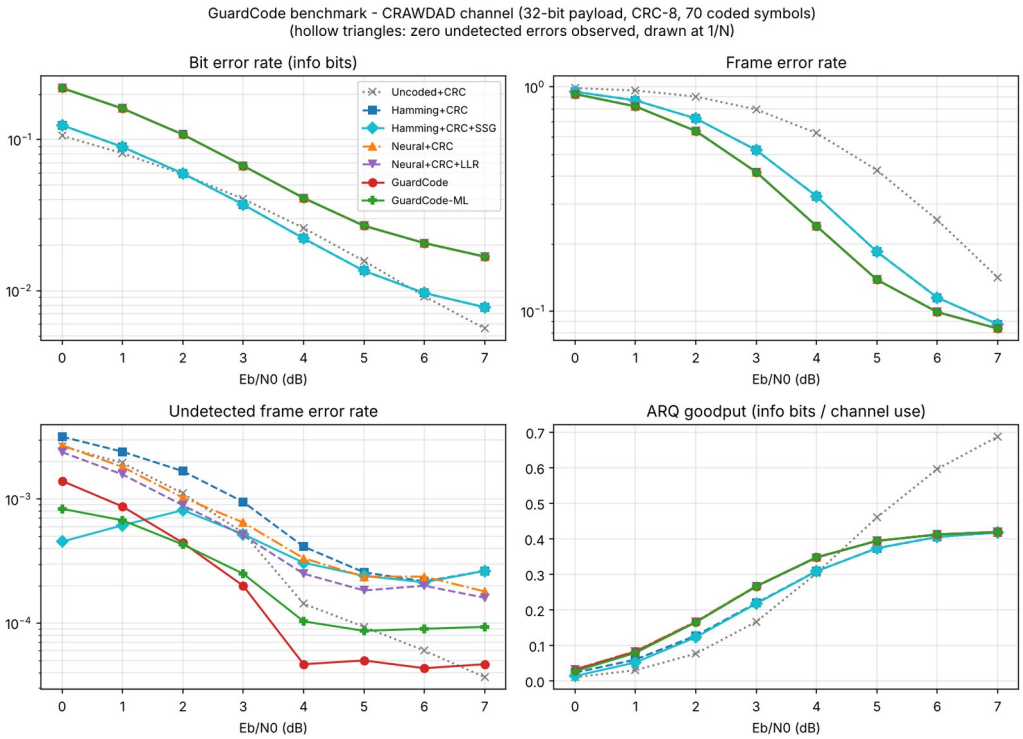


Figure 8 - Measured 802.15.4 channel (CRC-8)

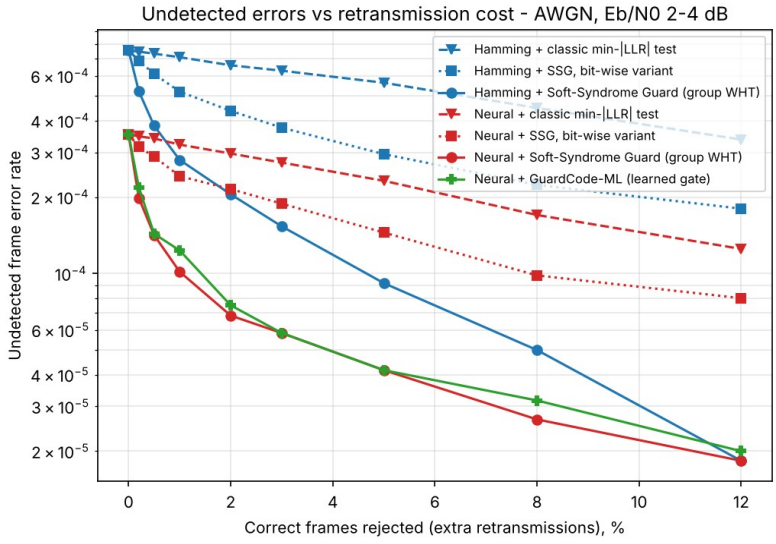


Figure 9 - Undetected error rate vs retransmission cost (AWGN, 2-4 dB)

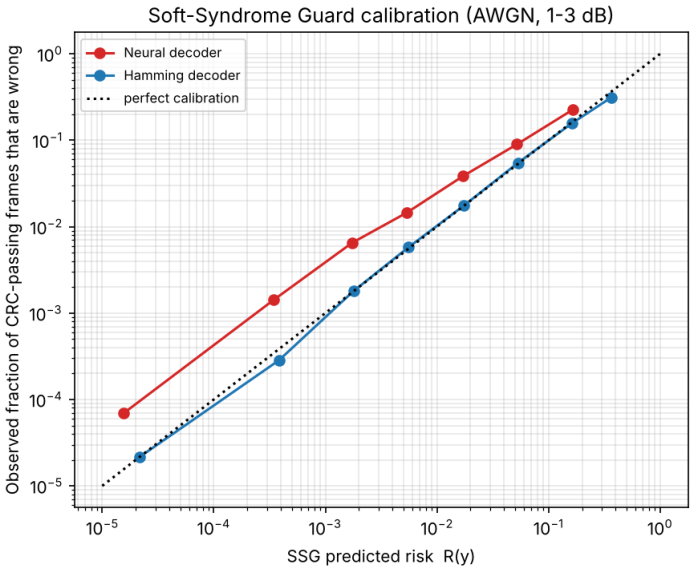


Figure 10 - Predicted risk vs observed error frequency

Calibration: the Hamming receiver's posteriors are exact on AWGN, so its points lie on the diagonal (934 undetected errors predicted vs 923 observed between 1 and 3 dB). The neural receiver is over-confident by a factor of about 2-4 (248 vs 534) but keeps the ordering of frames by risk, which is what the gate needs; this miscalibration is also why the learned gate helps.



Figure 11 - Profile switching when the channel changes AWGN -> burst -> AWGN (4 dB)

The controller switched 2 times and had the matching profile active in 100 % of the blocks; burst-phase goodput was 0.223 adaptive vs 0.208 with the fixed AWGN profile.

Negative result. A training-time version of the idea, a loss that penalises CRC-blind errors during training, did not change the share of undetected errors (0.106 %, 0.101 % and 0.099 % for $\lambda = 0$, 5 and 50). It is therefore not claimed.

7. Comparison Tables

All tables count undetected frames pooled over Eb/N0 = 0-7 dB, with every gate held to the same retransmission budget. Lower is better.

7.1 All receivers on all channels (CRC-8)

Receiver	AWGN	Burst	Measured 802.15.4	Goodput from 3 dB (AWGN)
Uncoded+CRC	1,817	3,006	1,975	0.470
Hamming+CRC	2,472	4,161	2,806	0.376
Hamming+CRC+SSG	486	2,135	1,027	0.376
Neural+CRC	1,559	3,229	2,154	0.405
Neural+CRC+LLR	1,428	3,054	1,846	0.403
GuardCode	742	1,531	929	0.404
GuardCode-ML	324	1,165	768	0.404

Table 8 - Undetected frames per receiver and channel, CRC-8

Comparison	AWGN	Burst	Measured 802.15.4
GuardCode vs plain CRC acceptance	52 %	53 %	57 %
GuardCode vs min-|LLR| test	48 %	50 %	50 %
GuardCode-ML vs GuardCode	56 %	24 %	17 %
GuardCode-ML vs min-|LLR| test	77 %	62 %	58 %
Hamming + SSG vs Hamming (decoder-agnostic)	80 %	49 %	63 %

Table 9 - Reduction in undetected frames

7.2 This work vs existing approaches

Method	Extra bits	Changes CRC	Works with any decoder	Undetected AWGN	Undetected burst	Undetected measured
Plain CRC acceptance (standard receiver)	0	No	Yes	1,559	3,229	2,154
Min-|LLR| confidence threshold (prior art)	0	No	Yes	1,428	3,054	1,846
Longer CRC (CRC-16 instead of CRC-8)	+8 per frame	Yes	Yes	5	17	13
Soft-Syndrome Guard (GuardCode, this work)	0	No	Yes	742	1,531	929
Learned gate (GuardCode-ML, this work)	0	No	Needs training	324	1,165	768

Table 10 - Comparison with existing approaches (neural decoder)

Only the longer CRC does better than GuardCode, and it pays with 8 extra bits on every frame: goodput from 3 dB upward falls from 0.40 to 0.29 information bits per channel use on AWGN. GuardCode and GuardCode-ML keep the frame and the CRC unchanged.

7.3 Detailed results per channel

Scheme	Frames	Frame errors	Undetected	% of errors undetected	Ledger estimate (accepted)
Uncoded+CRC	24,00,000	14,73,042	1,817	0.123	-
Hamming+CRC	24,00,000	10,15,487	2,472	0.243	-
Hamming+CRC+SSG	24,00,000	10,15,487	486	0.048	483
Neural+CRC	24,00,000	8,64,836	1,559	0.180	-
Neural+CRC+LLR	24,00,000	8,64,836	1,428	0.165	-
GuardCode	24,00,000	8,64,836	742	0.086	132
GuardCode-ML	24,00,000	8,64,836	324	0.037	-

Table 11 - AWGN channel

Scheme	Frames	Frame errors	Undetected	% of errors undetected	Ledger estimate (accepted)
Uncoded+CRC	24,00,000	18,44,067	3,006	0.163	-
Hamming+CRC	24,00,000	15,35,751	4,161	0.271	-
Hamming+CRC+SSG	24,00,000	15,35,751	2,135	0.139	284
Neural+CRC	24,00,000	13,77,323	3,229	0.234	-
Neural+CRC+LLR	24,00,000	13,77,323	3,054	0.222	-
GuardCode	24,00,000	13,77,323	1,531	0.111	1621
GuardCode-ML	24,00,000	13,77,323	1,165	0.085	-

Table 12 - Burst channel

Scheme	Frames	Frame errors	Undetected	% of errors undetected	Ledger estimate (accepted)
Uncoded+CRC	24,00,000	15,27,742	1,975	0.129	-
Hamming+CRC	24,00,000	11,33,274	2,806	0.248	-
Hamming+CRC+SSG	24,00,000	11,33,274	1,027	0.091	533
Neural+CRC	24,00,000	10,08,066	2,154	0.214	-
Neural+CRC+LLR	24,00,000	10,08,066	1,846	0.183	-
GuardCode	24,00,000	10,08,066	929	0.092	1538
GuardCode-ML	24,00,000	10,08,066	768	0.076	-

Table 13 - Measured 802.15.4 channel

Scheme	0 dB	1 dB	2 dB	3 dB	4 dB	5 dB	6 dB	7 dB
Uncoded+CRC	0.9871	0.9613	0.8992	0.7793	0.5975	0.3895	0.2070	0.0891
Hamming+CRC	0.9455	0.8581	0.6968	0.4767	0.2596	0.1072	0.0336	0.0073
Hamming+CRC+SSG	0.9455	0.8581	0.6968	0.4767	0.2596	0.1072	0.0336	0.0073
Neural+CRC	0.9182	0.7969	0.5922	0.3523	0.1574	0.0517	0.0120	0.0021
Neural+CRC+LLR	0.9182	0.7969	0.5922	0.3523	0.1574	0.0517	0.0120	0.0021

Scheme	0 dB	1 dB	2 dB	3 dB	4 dB	5 dB	6 dB	7 dB
GuardCode	0.9182	0.7969	0.5922	0.3523	0.1574	0.0517	0.0120	0.0021
GuardCode-ML	0.9182	0.7969	0.5922	0.3523	0.1574	0.0517	0.0120	0.0021

Table 14 - Frame error rate per Eb/N0, AWGN

The neural code beats Hamming(7,4) at the same rate: at 4 dB its FER is 0.157 vs 0.260, a gain of about 1 dB.

7.4 CRC-8 vs CRC-16

Receiver	CRC-8 AWGN	CRC-16 AWGN	CRC-8 burst	CRC-16 burst	CRC-8 measured	CRC-16 measured
Uncoded+CRC	1,817	49	3,006	65	1,975	51
Hamming+CRC	2,472	6	4,161	11	2,806	6
Hamming+CRC+SSG	486	0	2,135	2	1,027	3
Neural+CRC	1,559	5	3,229	17	2,154	13
Neural+CRC+LLR	1,428	5	3,054	16	1,846	13
GuardCode	742	0	1,531	8	929	3

Table 15 - Undetected frames with CRC-8 and CRC-16

CRC-16 removes about 99 % of CRC-8's undetected errors. The guard still lowers the CRC-16 remainder (burst 17 to 8, measured 13 to 3), but these single-digit counts show the direction rather than a precise percentage.

7.5 Trade-off at different retransmission budgets

Decoder (channel)	No gate	LLR @1 %	SSG @1 %	LLR @5 %	SSG @5 %
Neural (AWGN)	3.53e-4	3.22e-4	1.02e-4	2.32e-4	4.17e-5
Hamming (AWGN)	7.55e-4	7.10e-4	2.80e-4	5.63e-4	9.17e-5
Neural (BURST)	1.39e-3	1.32e-3	7.55e-4	1.03e-3	3.37e-4
Hamming (BURST)	1.99e-3	1.90e-3	1.44e-3	1.60e-3	8.80e-4

Table 16 - Undetected frame error rate at 1 % and 5 % false-reject budgets

8. Conclusion and Future Work

GuardCode shows that the CRC carries more information than a pass/fail bit. Combined with the decoder's soft output, the CRC's exact structure tells the receiver how likely a passing frame is to be wrong, and acting on that estimate removes about half the undetected errors left by the classic test, at no extra bandwidth and with the standard CRC unchanged. A learned gate removes more, and the result holds on a channel built from real 802.15.4 traces. The work also produced a neural code that beats Hamming(7,4) by about 1 dB, a live dashboard, a database of over a million frames, CRC-8 and CRC-16 support, and a draft patent with VIT invention disclosure forms.

Future work:

- CRC-24/32, where the exact 2^r-term sum must be replaced by a codeword-side sum or a list of the most likely alternative frames;
- LDPC, polar and convolutional decoders, and HARQ soft combining;

- the 802.11 traces of the CRAWDAD dataset, and slow fading within traces;
- an implementation on a software-defined-radio testbed (TRL 4).

References

1. B. A. Forouzan, Data Communications and Networking, 5th ed., McGraw-Hill.
2. C.-Y. Lou, B. Daneshrad, R. D. Wesel, "Convolutional-code-specific CRC code design," IEEE Trans. Communications, 2015.
3. "Autoencoders with list decoding for reliable data transmission over noisy channels," US Patent Application 2023/0198672 A1.
4. "Hierarchical deep channel coding," US Patent 12,155,482 B2.
5. Y. Jiang et al., "Turbo Autoencoder: Deep learning based channel codes for point-to-point communication channels," NeurIPS 2019.
6. A. R. Raghavan, C. W. Baum, "A reliability output Viterbi algorithm with applications to hybrid ARQ," IEEE Trans. Information Theory, 1998.
7. G. D. Forney, "Exponential error bounds for erasure, list, and decision feedback schemes," IEEE Trans. Information Theory, 1968.
8. R. G. Gallager, Low-Density Parity-Check Codes, MIT Press, 1963.
9. E. N. Gilbert, "Capacity of a burst-noise channel," BSTJ, 1960; E. O. Elliott, "Estimates of error rates for codes on burst-noise channels," BSTJ, 1963.
10. "Error detection strategies for CRC-concatenated polar codes under successive cancellation list decoding," DLR, 2023.
11. A. Iqbal, K. Shahzad, S. A. Khayam, Y. Cho, "CRAWDAD dataset niit/bit_errors (v. 2008-07-08)," IEEE DataPort, DOI 10.15783/C7S88K.
12. L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," Proceedings of the IEEE, 1989.
13. F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, 2011.

Appendix A - How to Run

```
Naresh_Computer_Networks_Project\
  setup.bat            one-click: venv, dependencies, benchmark import, 1M-frame
database
  run_dashboard.bat    live dashboard at http://127.0.0.1:5000  (explorer: /explorer)

Manual (inside .venv):
  python -m guardcode.evaluate --frames 300000                        CRC-8, AWGN +
burst
  python -m guardcode.evaluate --profiles crawdad                     measured
802.15.4 channel
  python -m guardcode.evaluate --crc CRC-16 --profiles awgn,burst,crawdad
  python scripts\fit_crawdad.py                                       refit the
CRAWDAD traces
  python scripts\train_mlguard.py --frames 6000000                    retrain
GuardCode-ML
  python scripts\train_all.py --crc CRC-16                            CRC-16 neural
codes
  python scripts\analysis.py ; python scripts\summarize.py
  python scripts\populate_db.py --frames 1000000
  python dashboard\app.py
```